Ders 24 Çalışma Özeti

Ders 24 Çalışma Özeti

Genel Konular

- Hoca, bu dersten itibaren kalan üç dersin konularının finalde (projede) sorumlu olunmayacak, ama ileride çok işe yarayacak konular olduğunu söyler. Bu derste işlenenler:
  - Sepet analizi (birliktelik analizi).
  - Makrolar.
  - Stringler.
  - Sezar şifreleme ve harf tablosuyla şifreleme.
  - Metinde birlikte geçen kelimeler.
  - Görüntüde gürültü temizlemek için ortalama filtresi.
  - Örnek bir proje raporu.
- Proje üzerine sorular
  - Bir öğrenci, geçen yılın YouTube'daki proje videolarında kendi kodlarından çok algoritmanın anlatıldığını söyledi. Hoca, beklentinin hem kodu hem algoritmayı anlatmak ve algoritmaya hâkim olmak olduğunu, başkalarının ne yaptığına değil proje metnine bakılması gerektiğini söyler.
  - Proje sisteme 13 Haziran 23.59'a kadar tanımlandı, gerekirse ileri alınabilir. Şimdiden yüklenebilir. Erken yükleyenlerin projeleri finallerden önce kontrol edilmeye başlanabilir.
  - Sayısal analiz projesinde birebir kontrol var. Bu derste olup olmayacağı Fatih Hoca ile konuşulacak.
  - Ders sonunda bir öğrencinin Diffie-Hellman anahtar değişimi projesi soruldu. Hoca, algoritmanın çalışma mantığını göstermek için küçük sayılarla (küçük aralıkta rastgele sayılarla) test etmenin yeterli olduğunu söyler. RSA'da da çok büyük asal sayıların büyük kuvvetleri hesaplanır, kaynaklarda bunlar küçük örneklerle gösterilir.
- Sepet analizi (association analysis, birliktelik analizi)
  - Müşterilerin alışveriş sepetleri (fişleri) analiz edilir. Ürünler marka yerine ürün grubu ID'siyle gösterilir. Bu bir veri madenciliği konusudur. Burada anlatılan algoritma verimli değildir, mantığı göstermek içindir.
  - Örnek: bir kitabı sepete atınca "bunu alanlar şunu da aldı" önerisi. Patatesle soğan, fizik kitabıyla matematik kitabı gibi alakalı birliktelikler olabilir. Daha ilginç olanı alakasız görünen birlikteliklerdir (patates alanların %98'inin çivi alması gibi). Klasik bir çalışmada bebek bezi alanların çoğunun bira da aldığı bulunmuştu.
  - Bu bir korelasyondur, nedensellik değildir. Patates almak çivi almaya sebep olmaz.
  - Hocanın doktora tezindeki müzik önerisi ise içerik tabanlı (content-based) bir öneridir: şarkıların akustik özelliklerine bakar. Sepet analizi ise insanların eğilimlerine bakar. Bir öğrencinin bahsettiği, sorularla akıldaki kişiyi tahmin eden asistanlar (Akinator gibi) da işbirlikçi filtreleme (collaborative filtering) ve öneri sistemleriyle ilgilidir.
- Sepet analizinin kodlanması (`Sepetv1.c`)
  - Her müşteri farklı sayıda ürün aldığı için müşteri × ürün listesi düzensiz (ragged/jagged) bir dizi olur. Bunun yerine her satırı (sepet ID, ürün ID) olan iki sütunlu bir matris kullanılır: `{1,2}, {2,1}, {2,4}, {2,2}, {1,4}, {3,3}, ….`
  - Bu liste karışıktır. Sepet numarasına göre sıralamak da istenen bilgiyi vermez, çünkü amaç müşteriler değil, birlikte alınan ürünlerdir.
  - Önce sepet × ürün boyutunda 0-1 bir `A` matrisi oluşturulur: `A[sepet[i][0]][sepet[i][1]] = 1;`. Tek döngü yeterlidir, çünkü listenin yalnızca iki sütunu var.

  - Matrisi sıfırlamak için `static` yerine `= {0}` (veya `{{0}}`) yazılmalıdır. `static`'in başka kullanım amaçları vardır ve bilinçsiz kullanılırsa yanlış sonuçlara yol açar.
  - Sonra ürün × ürün boyutunda bir `T` matrisi oluşturulur. `T[i][j]`, `i` ve `j` ürünlerinin kaç sepette birlikte alındığıdır. `i` ürünler, `j` `i+1`'den ürünler (sıra önemsiz), `k` sepetler üzerinde döner:
    * İlk sürüm: `if (A[k][i] == 1 && A[k][j] == 1) T[i][j]++;`.
    * `if`'siz sürüm: değerler 0-1 olduğu için çarpım yeterli, `T[i][j] += A[k][i] * A[k][j];`.
  - Derste ilk denemede indisler ters yazıldığı için sonuç yanlış çıktı, `A[k][i]` olarak düzeltildi.
  - `T`'nin en büyük elemanı en çok birlikte satılan ürün çiftini verir. Örnekte 2. ve 4. ürünler iki sepette birlikte alınmış.
  - Çalışma sorusu: en çok birlikte satılan üçlü ürünler. Bunun için üç boyutlu bir matris (üst üste duran `k` tane matris) gerekir. Pazartesi günü kodlanacak.
  - Karmaşıklık: ürün sayısı n ise iç içe ürün döngüleri $\frac{n(n-1)}{2}$ kez döner. Sepet sayısı da yaklaşık n ise toplam n^3 mertebesindedir. Bellek: `A` için ürün × sepet ve `T` için ürün × ürün, yani iki tane n^2'lik yer. Daha iyi teknikler veri madenciliği dersinde görülür.
- Makrolar
  - `#define` yalnızca sabit tanımlamak için değil, fonksiyon benzeri makrolar için de kullanılır: `#define MYSQR(x) x*x`. Makro bir fonksiyon değildir. Derlemeden önce önişlemci tarafından metin olarak yerine konur (substitution). Hoca bunu "fonksiyona biz gideriz, makro bize gelir" diye özetler.
  - Fonksiyon çağrısında olduğu gibi stack'e yedekleme yapılmaz, bu yüzden kısa işler için daha hızlıdır.
  - Yan etki: `MYSQR(5-3)`, `5-3*5-3` olur ve sonuç 4 değil -13'tür. Bunu önlemek için her parametre ayrı ayrı ve bütün ifade parantez içine alınır: `#define MYSQR1(x) ((x)*(x))`. Çağırırken parametreyi paranteze almak da çalışır ama doğru yol tanımdaki parantezdir.
  - Birden fazla parametre de olabilir: `#define MIN(x,y) ((x)<(y)?(x):(y))`. Tanımda `=` ve sonda `;` kullanılmaz.
  - Kısa, tek satırlık işler için makro, uzun işler (ör. matris yazdırma) için fonksiyon kullanılır. Makroların ayrıntısı gelecek dönem Yapısal Programlama dersinde görülecek.
- Stringler
  - C'de string diye bir veri tipi yoktur. String, sonunda `'\0'` (null) karakteri olan bir karakter dizisidir. `char name[50] = "deneme";` 50 karakterlik yerin ilk 6 karakterine "deneme"yi, 7.'sine `'\0'`'ı yazar.
  - `strlen` (`<string.h>`) 6 verir. Kendi yazdığımız uzunluk döngüsü: `while (name[i] != '\0') i++;`.
  - `while (name[i++]);` 7 verir, çünkü `i++` önce değeri kullanır sonra artırır ve `'\0'`'ı kontrol ettikten sonra da `i` bir kez artar. `i++` ile `++i` farkı: biri önce kullanır sonra artırır, diğeri önce artırır sonra kullanır. Vizede bununla ilgili bir soru vardı.
  - Aynı işi yapan iki sayaç (`i` ve `t`) tutmak gereksizdir.
  - Büyük harfi küçüğe çevirmek: ASCII'de büyük harfler 65'ten, küçük harfler 97'den başlar, fark 32'dir. `name[i] += 32;`. Derste ilk denemede döngüde `i++` unutulduğu için yalnızca ilk karakter defalarca değişti.
  - `printf("%s", name)` dizinin başından `'\0'`'a kadar yazar.
- Sezar şifreleme
  - Her karaktere sabit bir sayı eklenir: $y = x + c$. Örnek: `c = 3` ile "abcd", "defg" olur.
  - Şifreli metin ayrı bir diziye yazılır: `name2[i] = name[i] + c;`. Döngüden sonra `name2[i] = '\0';` yazılmalıdır. Yazılmazsa sonunda çöp karakterler görülebilir.
  - Çözme fonksiyonun tersidir: `name3[i] = name2[i] - c;`. Sonuç yeniden "abcd" oldu.
  - Afin şifreleme: $y = ax + b$. Şifreleme her zaman çalışır ama çözme her zaman mümkün değildir. Nedeni pazartesi konuşulacak.
  - Tabloyla şifreleme: bütün karakterler bir matrise yerleştirilir. Her harf, tablodaki satır ve sütun numarasıyla (ör. `satır * 10 + sütun`) kodlanır. Çözerken sayıdan satır ve sütun bulunur.
  - Tablo elde değilse şifre harf sıklığıyla kırılabilir: şifreli metinde en çok tekrar eden sayı, o dilde en çok kullanılan harfe karşılık gelir. Türkçede en sık harf A (sonra E, İ, N, R, L), İngilizcede E'dir. Bunu zorlaştırmak için tabloya her harf o dildeki sıklığıyla

orantılı sayıda konur ve şifrelerken harfin kodlarından biri rastgele seçilir.

- Metinde birlikte geçen kelimeler
  - Metin, kelime ID'lerinden oluşan `n` elemanlı bir dizi olarak tutulur. `W` kelime sayısı × kelime sayısı boyutunda bir matristir. `W[M[i]][M[i+1]]++` ile hangi kelimenin hangisinden hemen sonra geldiği sayılır. Benzer şekilde bir kelimenin `k` öncesindeki ve sonrasındaki kelimeler de sayılabilir.
  - Bu, doğal dil işleme ve metin madenciliğinde kullanılır. Örnek: deasciification, yani Türkçe karakterleri kaybolmuş bir metni ("Sisli Caddesi") bağlama göre düzeltmek ("Şişli Caddesi"). PDF'ten Word'e dönüştürmede bozulan karakterler de böyle düzeltilebilir. Birlikte geçen kelimeler anlamca birbirine yakındır.
- Görüntüde gürültü temizleme: ortalama filtresi
  - Gri seviyeli 8 bitlik bir resim (0-255) bir matris olarak okunur. Resimde komşu pikseller birbirine yakın değerlerdedir. Arada bir 255 veya 0 piksel gürültüdür (salt and pepper).
  - Ortalama filtresi (mean filter): her piksel, çevresindeki 3 × 3 (veya 5 × 5, 9 × 9) pencerenin ortalamasıyla değiştirilir. `i` ve `j` resmi `n - filtre` sınırına kadar, `m` ve `n` filtre penceresini gezer. `B[i][j] = toplam / (filtre * filtre)`. Resmin okunup yazılması hazır kodla yapıldı, bu dersin konusu değil.
  - Sonuç: 3 × 3 filtre gürültüyü kısmen azalttı. 9 × 9 filtre gürültüyü daha çok azalttı ama resmi iyice bulanıklaştırdı. Gürültü de ortalamaya dahil olduğu için ortalama filtresi iyi bir çözüm değildir. Daha iyi bir filtre pazartesi anlatılacak. Bir öğrenci standart sapmanın dışında kalan pikselleri ayıklamayı önerdi.
- Örnek proje raporu
  - Kapak sayfası (ad, soyad, numara, kullanılan e-posta, ödev adı, yürütücü, tarih), içindekiler (Word'de kolayca oluşturulur), bölümler.
  - Şekil yazıları şeklin altına, tablo yazıları tablonun üstüne yazılır. Denklemler açıklanır.
  - Adı, numarası olmayan, özensiz raporlar kabul görmez. Yarım saat emek düzgün bir rapor için yeterlidir.
  - Hoca öğrencilerden dönem değerlendirmesi için bir anket formunu doldurmalarını ister.

Hocanın Özellikle Vurguladığı Kısımlar

- Günlük hayatta gördüğünüz yazılımları (öneri listeleri gibi) mesleğinizle ilişkilendirin.
- Birliktelik korelasyondur, nedensellik değildir.
- Makro parametrelerini ve bütün ifadeyi paranteze alın.
- Yeni oluşturduğunuz string'in sonuna `'\0'` koymayı unutmayın.
- Bilmediğiniz bir özelliği (`static` gibi) kullanmayın.
- Raporunuz özenli olsun ve kaynaklarınızı belirtin.

Kısa Tekrar Notları

- Sepet analizi: (sepet, ürün) listesi, 0-1 sepet × ürün matrisi `A`, birlikte satış matrisi `T[i][j] += A[k][i] * A[k][j]`. Karmaşıklık n^3, bellek $2n^2$.
- `#define MYSQR(x) ((x)*(x))`. Parantezsiz `MYSQR(5-3)` -13 verir.
- String = `char` dizisi + `'\0'`. `strlen("deneme") = 6`.
- Büyük-küçük harf farkı 32.
- Sezar: şifrele `+c`, çöz `-c`.
- Ortalama filtresi: pencere ortalaması. Büyük pencere resmi bulanıklaştırır.

Detaylı Açıklamalar

Derste yazılan kodların denenmesi

Sepet analizi, makrolar ve Sezar şifreleme ders sonrasında bir programda toplandı ve `gcc -Wall -std=c89 -pedantic` ile denendi:

```
#define MYSQR(x) x*x
#define MYSQR1(x) ((x)*(x))
```

```
for (i = 0; i < n; i++) A[sepet[i][0]][sepet[i][1]] = 1;
for (i = 0; i < urun; i++)
    for (j = i + 1; j < urun; j++)
        for (k = 0; k < sepetSay; k++)
            T[i][j] += A[k][i] * A[k][j];

i = 0; while (s[i]) { name2[i] = s[i] + c; i++; } name2[i] = '\0';
i = 0; while (name2[i]) { name3[i] = name2[i] - c; i++; } name3[i] = '\0';
```

- Derste son kullanılan 7 kayıtlık sepet listesiyle `T[2][4]` 2, `T[1][2]`, `T[1][4]` ve `T[2][3]` 1 çıktı. 2. ve 4. ürünlerin en çok birlikte satıldığı sonucu dersle aynıdır.
- `MYSQR(5-3)` -13, `MYSQR1(5-3)` 4, `strlen("deneme")` 6, `while (name[i++]);` sonrası `i` 7 verdi.
- "abcd", `c = 3` ile "defg" oldu ve geri "abcd"ye çözüldü.

Ders sonrası notlar

- **Çöp karakter neden görülmedi?** Hoca `'\0'` eklemeden çöp değer göstermek istedi ama göremedi. Sebebi, `name2`'nin `char name2[50] = "";` ile tanımlanmasıdır. Bir dizi kısmen ilk değerle tanımlanınca kalan bütün elemanlar 0 olur. Yani `name2`'nin her yeri zaten `'\0'`'dı. İlk değer verilmemiş yerel bir dizide (`char name2[50];`) aynı kodla çöp karakterler görülebilir. Bu yüzden `'\0'`'ı açıkça yazmak doğru alışkanlıktır.
- **Sezar şifrelemede alfabe sınırı:** `c = 3` ile 'x', 'y', 'z' harfleri alfabenin dışına çıkar ('z' + 3 = '}'). Klasik Sezar şifrelemede alfabe döngüseldir: `(x - 'a' + c) % 26 + 'a'`. Çözerken `(y - 'a' - c + 26) % 26 + 'a'` kullanılır.
- **Afin şifrelemenin çözülmesi:** Hoca, $y = ax + b$'nin tersinin $\frac{b}{a}$ gibi bir şey olduğunu söyledi. 26 harflik bir alfabede şifreleme $y = (ax + b) \bmod 26$, çözme $x = a^{-1}(y - b) \bmod 26$'dır. Burada a^{-1}, a'nın 26'ya göre tersidir ($a \cdot a^{-1} \equiv 1 \pmod{26}$). Bu ters yalnızca $\gcd(a, 26) = 1$ ise vardır. Örneğin $a = 2$ ile 'a' ve 'n' aynı harfe dönüşür ve metin geri çözülemez. Derste "çözme her zaman çalışmaz" denmesinin sebebi budur.
- **Karakter tablosunun boyutu:** Hoca 256 karakterlik ASCII tablosu için "en az 8 × 8'lik bir matris" dedi. 8 × 8 = 64 hücredir. 256 karakter için en az 16 × 16 gerekir. Standart ASCII 128 karakterdir. 256'ya genişletilmiş kod sayfalarıyla çıkılır.
- **`MIN(3, 3.6)` sonucu:** Derste makroya 3 ve 3.6 verilince anlamsız bir sonuç çıktı. İfadenin sonucu `double` olduğu halde `%d` ile yazdırıldı. Bu, `printf`'te tip uyuşmazlığıdır ve tanımsız davranıştır. `%f` ile yazdırılmalıdır.
- **Tuz-biber gürültüsü için daha iyi filtre:** Bu tür gürültüde pencerenin ortalaması yerine ortancası (medyanı) alınır (median filter). Gürültülü uç değerler ortanca değeri etkilemediği için kenarlar korunur ve resim bulanıklaşmaz.
- **Filtrede kenarlar:** Döngüler `n - filtre` sınırında bittiği için resmin son satır ve sütunları işlenmez. Ayrıca sonuç pencerenin merkezine değil sol üst köşesine yazılır. Bu yüzden resim yarım pencere kadar kayar. Pencere merkezli yazmak için `i` ve `j` `filtre / 2`'den başlatılıp pencere `i - filtre/2`'den `i + filtre/2`'ye gezdirilir.
- **Not:** İsterseniz bu dersin altyazı (.srt) dosyasını Gemini Notebook gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.